



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Instituto de Ciências Exatas e de Informática

Relatório de Implementação e Comparação de Algoritmos de Busca para o 8-Puzzle

Felipe Gabriel de Carvalho
Gustavo Henrique dos Santos Dias
João Lucas de Melo Quintão
Raphael Oliveira de Araujo
Thiago Henrique Gomes Feliciano

Resumo

O 8-puzzle é um problema clássico de busca em espaço de estados amplamente utilizado na avaliação e comparação de algoritmos de Inteligência Artificial. Este trabalho tem como objetivo implementar e comparar diferentes algoritmos de busca para a resolução desse problema, analisando seu desempenho quanto ao tempo de execução, número de nós visitados e qualidade das soluções encontradas. Foram implementados os algoritmos Busca em Largura (BFS), Busca em Profundidade (DFS), Busca de Custo Uniforme, Busca Gulosa e A*, sendo os algoritmos de busca informada avaliados com as heurísticas de Hamming, Distância de Manhattan e Conflito Linear. Os experimentos foram realizados sobre um conjunto de instâncias do 8-puzzle, permitindo uma análise comparativa entre as diferentes estratégias de busca. Os resultados demonstraram que o algoritmo A* combinado com a heurística de Conflito Linear apresentou o melhor equilíbrio entre eficiência e qualidade da solução, reduzindo significativamente o número de estados explorados e o tempo de execução, ao mesmo tempo em que preservou a otimalidade. Por outro lado, a Busca Gulosa obteve os menores tempos de execução, porém sem garantir soluções ótimas, enquanto o DFS apresentou o pior desempenho geral. Os resultados evidenciam a influência da escolha da heurística no desempenho dos algoritmos de busca informada e reforçam a superioridade do A* com heurísticas mais informativas para a resolução do 8-puzzle.

Palavras-chave: 8-puzzle; algoritmos de busca; inteligência artificial; A*; heurísticas; BFS; DFS; dijkstra.

Abstract

The 8-puzzle is a classic state-space search problem widely used in evaluating and comparing Artificial Intelligence algorithms. This work aims to implement and compare different search algorithms to solve this problem, analyzing their performance in terms of execution time, number of nodes visited, and quality of the solutions found. The algorithms implemented were Breadth-First Search (BFS), Depth-First Search (DFS), Uniform-Cost Search, Greedy Search, and A*, with the informed search algorithms evaluated using the Hamming, Manhattan Distance, and Linear Conflict heuristics. The experiments were conducted on a set of 8-puzzle instances, allowing a comparative analysis between the different search strategies. On the other hand, Greedy Search had the shortest execution times, but it didn't guarantee optimal solutions, while DFS showed the worst overall performance. The results highlight how the choice of heuristic affects the performance of informed search algorithms and reinforce the superiority of A* with more informative heuristics for solving the 8-puzzle.

Keywords: 8-puzzle; search algorithms; artificial intelligence; A*; heuristics; BFS; DFS; dijkstra.

1 INTRODUÇÃO

O **8-puzzle** é um quebra-cabeça composto por um tabuleiro 3×3 contendo 8 peças numeradas (de 1 a 8) e um espaço vazio. O objetivo é, a partir de uma configuração inicial qualquer, alcançar uma configuração final (objetivo) movendo as peças adjacentes ao espaço vazio, uma por vez, até reorganizá-las na ordem correta.

Esse problema é um exemplo clássico em Inteligência Artificial para ilustrar problemas de busca em espaço de estados, pois pode ser formalmente definido pelos seguintes componentes:

- **Estado inicial:** É o estado em que o agente começa. A configuração de partida do tabuleiro, isto é, a disposição específica das 8 peças e do espaço vazio nas 9 posições do tabuleiro 3×3 .
- **Conjunto de ações disponíveis:** Os movimentos possíveis do espaço vazio: para cima, baixo, esquerda ou direita, equivalentes a mover a peça adjacente para a posição vazia. No máximo 4 ações estão disponíveis em cada estado, variando conforme a posição do espaço vazio no tabuleiro.
- **Modelo de transição:** Dado um estado e uma ação, retorna o estado resultante após mover o espaço vazio na direção indicada. O estado inicial, as ações e o modelo de transição definem o espaço de estados do problema, ou seja, todos os estados atingíveis a partir do estado inicial.
- **Teste do objetivo:** Estado explícito: a configuração final desejada, geralmente as peças organizadas em ordem crescente (1 a 8) com o espaço vazio em uma posição fixa.
- **Custo da solução:** Cada movimento tem custo unitário (custo 1 por passo), de modo que o custo do caminho é a soma dos custos dos passos individuais, equivalente ao número total de movimentos realizados. A solução ótima é aquela cujo caminho tem o menor número de movimentos entre o estado inicial e o estado objetivo.

O espaço de estados do 8-puzzle possui 181.440 estados alcançáveis (metade das $9! = 362.880$ permutações possíveis, já que apenas configurações com paridade compatível ao objetivo são solucionáveis). Esse tamanho moderado faz do 8-puzzle um problema ideal para testar e comparar algoritmos de busca, como busca em largura (BFS), busca em profundidade (DFS), busca de custo uniforme e, principalmente, algoritmos heurísticos como o A* e a busca gulosa, que utiliza estimativas (como a distância de Hamming, custo uniforme e distância de Manhattan) para guiar a busca de forma mais eficiente até a solução.

2 MODELAGEM

Esta seção apresenta a formulação explícita do 8-puzzle como um problema de busca, definindo formalmente o estado inicial, o conjunto de ações, o modelo de transição, o teste do objetivo e o custo da solução.

2.1 Estado inicial

O estado inicial é representado por uma configuração específica do tabuleiro 3×3 , na qual as 8 peças numeradas (1 a 8) e o espaço vazio (representado por 0) estão distribuídos em alguma ordem entre as 9 posições disponíveis. Formalmente, um estado pode ser representado como uma matriz 3×3 ou como um vetor de 9 posições, por exemplo:

$$\text{Estado inicial} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 0 & 6 \\ 7 & 5 & 8 \end{pmatrix}$$

em que o valor 0 indica a posição do espaço vazio. Qualquer configuração das peças pode ser usada como estado inicial, desde que seja solucionável, ou seja, que possua a mesma paridade de permutação do estado objetivo.

2.2 Conjunto de ações

O conjunto de ações disponíveis corresponde aos movimentos possíveis do espaço vazio dentro do tabuleiro: CIMA, BAIXO, ESQUERDA e DIREITA. Cada ação move o espaço vazio na direção indicada, o que é equivalente a deslocar a peça adjacente para a posição que estava vazia.

O número de ações disponíveis em um determinado estado varia entre 2 e 4, dependendo da posição do espaço vazio no tabuleiro: nos cantos, apenas 2 ações são possíveis; nas bordas (não cantos), 3 ações; e no centro, todas as 4 ações estão disponíveis.

2.3 Modelo de transição

O modelo de transição é definido por uma função sucessora $\text{Resultado}(s, a)$, que recebe um estado s e uma ação a , e retorna o estado resultante após a aplicação dessa ação, isto é, o estado com o espaço vazio e a peça adjacente trocados de posição.

Dado o exemplo de estado inicial apresentado anteriormente, a aplicação da ação CIMA (que move o espaço vazio para cima, trocando-o com a peça que está imediatamente acima dele)

resultaria no estado:

$$\text{Resultado}(s, \text{CIMA}) = \begin{pmatrix} 1 & 0 & 3 \\ 4 & 2 & 6 \\ 7 & 5 & 8 \end{pmatrix}$$

O conjunto de pares $\langle \text{ação}, \text{estado} \rangle$ gerados pela função sucessora, aplicados recursivamente a partir do estado inicial, definem o espaço de estados do problema — representado como um grafo em que cada nó é um estado e cada aresta corresponde a uma ação que conecta dois estados. No caso do 8-puzzle, esse espaço de estados possui 181.440 estados alcançáveis a partir de qualquer estado inicial solucionável.

2.4 Teste do objetivo

O teste do objetivo verifica se o estado atual corresponde ao estado objetivo, definido explicitamente como a configuração em que as peças estão organizadas em ordem crescente, com o espaço vazio na primeira posição:

$$\text{Estado objetivo} = \begin{pmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{pmatrix}$$

O teste é realizado por uma simples comparação entre o estado atual e essa configuração-alvo, sendo positivo apenas quando todas as 9 posições coincidirem exatamente.

2.5 Custo da solução

Cada ação (movimento do espaço vazio) possui custo unitário, ou seja, custo igual a 1. Dessa forma, o custo total de um caminho entre o estado inicial e o estado objetivo é dado pela soma dos custos de cada passo individual, o que equivale, na prática, ao número total de movimentos realizados.

A solução ótima do problema é aquela que atinge o estado objetivo através do caminho de menor custo, ou seja, com o menor número de movimentos possível. É justamente essa definição de custo uniforme por passo que permite que algoritmos como BFS e Custo Uniforme garantam a otimalidade da solução encontrada, e que torna heurísticas como a distância de Hamming e a distância de Manhattan admissíveis, já que ambas nunca sobrestimam o número mínimo de movimentos necessários para alcançar o objetivo.

3 DESCRIÇÃO DOS ALGORITMOS

Esta seção apresenta os algoritmos de busca utilizados para resolver o 8-puzzle. Cada algoritmo explora o espaço de estados do problema de uma forma distinta, alternando entre estratégias de busca sem informação (que não utilizam conhecimento sobre o quão próximo um estado está do objetivo) e estratégias de busca com informação (que utilizam funções heurísticas para guiar a expansão dos estados de forma mais eficiente).

São descritos a seguir os algoritmos de Busca em Largura (BFS), Busca em Profundidade (DFS) e Custo Uniforme, classificados como buscas sem informação, além da Busca Gulosa e do algoritmo A*, que se utilizam de heurísticas para direcionar a busca rumo à solução com menor esforço computacional.

3.1 Algoritmo A*

O algoritmo A* é um método de busca informada que combina as vantagens da Busca de Custo Uniforme e da Busca Gulosa. A cada estado n , o A* calcula uma função de avaliação $f(n) = g(n) + h(n)$, em que $g(n)$ é o custo do caminho já percorrido desde o estado inicial até n , e $h(n)$ é uma estimativa heurística do custo restante até o estado objetivo. O algoritmo expande sempre o estado de menor valor de $f(n)$, utilizando uma fila de prioridade.

Quando a heurística utilizada é admissível (nunca sobrestima o custo real até o objetivo) e consistente, o A* garante encontrar a solução ótima, sendo, além disso, completo. Nesse trabalho, foram utilizadas três heurísticas para guiar o A* neste trabalho foram distância de Hamming, distância de Manhattan e Conflito Linear.

Devido à utilização de uma heurística admissível, o A* tende a expandir um número significativamente menor de estados em comparação às buscas não informadas, sendo, na prática, um dos algoritmos mais eficientes para a resolução do 8-puzzle. Sua complexidade de tempo e espaço, no pior caso, permanece $O(b^d)$, mas o número de estados efetivamente expandidos é, em geral, muito menor, sobretudo ao utilizar a distância de Manhattan.

3.2 Busca Gulosa

A Busca Gulosa (*Greedy Best-First Search*) é um algoritmo de busca informada que expande, a cada passo, o estado que parece estar mais próximo do objetivo, segundo uma função heurística $h(n)$, sem considerar o custo do caminho já percorrido até aquele estado.

Essa estratégia tende a encontrar soluções rapidamente quando a heurística é informativa, mas não garante encontrar o caminho de menor custo, podendo seguir por ramos que parecem promissores localmente, mas que levam a soluções mais longas. Além disso, não é garantidamente completa em espaços de estados muito grandes, pois pode entrar em ciclos caso

não haja controle de estados visitados. A complexidade de tempo e espaço, no pior caso, é $O(b^d)$, embora na prática a busca gulosa costume expandir muito menos estados que as buscas não informadas, dependendo da qualidade da heurística utilizada.

3.3 Busca em Largura(BFS)

A Busca em Largura (*Breadth-First Search*) é um algoritmo de busca não informada que explora o espaço de estados nível por nível, expandindo primeiro todos os estados a uma distância d do estado inicial antes de avançar para a distância $d + 1$. Para isso, utiliza uma fila do tipo FIFO (First In, First Out) para armazenar os estados de fronteira, garantindo que os estados sejam visitados na ordem em que foram gerados.

Como todos os movimentos no 8-puzzle possuem custo unitário, o BFS garante encontrar a solução ótima (a de menor número de movimentos), sendo completo e ótimo. Sua principal desvantagem é o consumo de memória, já que armazena todos os estados de uma mesma camada simultaneamente. A complexidade de tempo e espaço é $O(b^d)$, em que b é o fator de ramificação (no máximo 4, no caso do 8-puzzle) e d é a profundidade da solução.

3.4 Busca em Profundidade(DFS)

A Busca em Profundidade (*Depth-First Search*) explora o espaço de estados expandindo o ramo mais profundo da árvore de busca antes de retroceder (backtracking) e explorar ramos alternativos. Utiliza uma pilha para controlar os estados a serem expandidos.

No contexto do 8-puzzle, o DFS apresenta uma limitação importante: como o espaço de estados pode conter ciclos e caminhos muito longos (ou infinitos, se não houver controle de estados visitados, por isso não é considerada completa), o algoritmo não garante encontrar a solução ótima, podendo até divergir em ramos sem solução. Por isso, costuma-se aplicar um limite de profundidade para evitar esse problema. Sua vantagem é o baixo consumo de memória, com complexidade de espaço $O(bd)$, sendo b o fator de ramificação e d a profundidade máxima explorada. Já a complexidade de tempo é $O(b^d)$ no pior caso.

3.5 Custo Uniforme

A Busca de Custo Uniforme (*Uniform Cost Search*) é uma generalização do BFS que expande sempre o estado de menor custo acumulado de caminho $g(n)$, em vez de expandir por nível. Para isso, utiliza uma fila de prioridade ordenada pelo custo do caminho percorrido até cada estado.

No 8-puzzle, como todos os movimentos possuem custo igual a 1, a Busca de Custo

Uniforme se comporta de maneira equivalente ao BFS, expandindo os estados também em ordem de profundidade crescente e garantindo encontrar a solução ótima. Sua complexidade de tempo e espaço também é $O(b^d)$, podendo ser mais custosa na prática devido ao gerenciamento da fila de prioridade.

4 HEURÍSTICAS

As heurísticas desempenham um papel fundamental nos algoritmos de busca informada, como a Busca Gulosa e o A*, pois fornecem uma estimativa $h(n)$ do custo necessário para alcançar o estado objetivo a partir de um estado n , orientando a busca de forma mais eficiente. Para que o A* garanta encontrar a solução ótima, é necessário que a heurística utilizada seja admissível, isto é, que nunca sobrestime o custo real restante até o objetivo ($h(n) \leq h^*(n)$, em que $h^*(n)$ é o custo real do caminho mais curto até a solução).

Nesta seção, são definidas as três heurísticas utilizadas neste trabalho para guiar o A*: a distância de Hamming, a distância de Manhattan e o Conflito Linear.

4.1 Distância de Hamming

A distância de Hamming, também conhecida como número de peças fora do lugar, conta quantas peças do tabuleiro não estão na posição correspondente ao estado objetivo, desconsiderando o espaço vazio.

Essa heurística é admissível, pois cada peça fora do lugar necessita de, no mínimo, um movimento para ser corrigida, de modo que o valor estimado nunca supera o número real de movimentos necessários. Entretanto, é a heurística menos informativa entre as três, já que não leva em consideração a distância real que cada peça precisa percorrer até sua posição correta, apenas se ela está ou não no lugar certo. Por esse motivo, tende a gerar um número maior de estados expandidos durante a busca, em comparação às demais heurísticas.

4.2 Distância de Manhattan

A distância de Manhattan soma, para cada peça fora do lugar, a distância mínima (em número de movimentos horizontais e verticais) entre sua posição atual e sua posição correspondente no estado objetivo, desconsiderando obstáculos ou outras peças no caminho.

Essa heurística também é admissível, pois a distância de Manhattan representa o número mínimo de movimentos que uma peça precisaria realizar para chegar à sua posição correta caso estivesse isolada no tabuleiro, sem considerar a interferência de outras peças — e essa interferência só pode aumentar o número real de movimentos necessários, nunca diminuí-lo. Por

considerar a distância real de deslocamento de cada peça, a distância de Manhattan é mais informativa que a distância de Hamming, levando geralmente a uma quantidade significativamente menor de estados expandidos.

4.3 Conflito Linear

O Conflito Linear é um refinamento da distância de Manhattan que identifica situações em que duas peças estão na mesma linha ou coluna de suas posições objetivo, mas em ordem invertida uma em relação à outra. Nesses casos, ao menos uma das peças precisa se desviar de sua linha ou coluna para permitir a passagem da outra, o que exige dois movimentos adicionais além dos já contabilizados pela distância de Manhattan.

A heurística é calculada somando-se a distância de Manhattan ao dobro do número de conflitos lineares identificados no tabuleiro. O Conflito Linear permanece admissível, pois os movimentos extras contabilizados correspondem a deslocamentos que de fato precisarão ser realizados para resolver o conflito entre as peças invertidas, não havendo sobrestimação do custo real da solução. Por incorporar essa informação adicional, o Conflito Linear é a heurística mais informativa das três, sendo dominante em relação à distância de Manhattan (isto é, $h_{\text{Manhattan}}(n) \leq h_{\text{Conflito Linear}}(n) \leq h^*(n)$ para todo estado n), o que resulta, na prática, no menor número de estados expandidos entre as heurísticas estudadas.

5 RESULTADOS EXPERIMENTAIS

Esta seção apresenta os resultados experimentais obtidos para cada algoritmo de busca implementado, executados sobre um conjunto de 9 instâncias de teste do 8-puzzle. Para cada algoritmo/heurística, foram analisados o número médio de movimentos da solução (equivalente à profundidade do nó objetivo), o número médio de nós visitados (nós expandidos/gerados durante a busca) e o tempo de execução em milissegundos (médio, mínimo e máximo).

Algoritmo	Heurística	Testes	Movimentos médios	Nós visitados médios	Tempo médio (ms)	Menor tempo (ms)	Maior tempo (ms)
BFS	Não se aplica	9	19,00	75226,56	228,93	0,06	540,29
DFS	Não se aplica	9	86379,89	131834,33	419,33	1,14	569,20
Custo Uniforme	Não se aplica	9	19,00	73171,22	466,85	0,08	1123,01
A*	Hamming	9	19,00	12402,22	90,62	1,80	344,25
A*	Manhattan	9	19,00	1088,67	7,61	0,58	29,35
A*	Conflito Linear	9	19,00	572,89	5,32	0,45	14,06
Busca Gulosa	Hamming	9	62,78	311,22	2,01	0,47	6,56
Busca Gulosa	Manhattan	9	45,44	176,44	1,46	0,39	2,45
Busca Gulosa	Conflito Linear	9	28,33	55,22	1,00	0,56	1,53

Figura 1 – Resultados experimentais médios por algoritmo/heurística

A partir dos dados apresentados na Figura 1, observa-se que BFS e Custo Uniforme obtiveram exatamente a mesma profundidade média de solução (19,00 movimentos), o que era esperado, já que ambos garantem a solução ótima quando o custo de cada movimento é

unitário. O A*, independentemente da heurística utilizada, também encontrou soluções com profundidade média de 19,00, confirmando a propriedade de otimalidade do algoritmo quando guiado por heurísticas admissíveis. Já o DFS apresentou um número médio de movimentos extremamente elevado (86379,89), evidenciando que esse algoritmo não garante a solução ótima, podendo seguir por caminhos muito mais longos antes de encontrar o objetivo.

Quanto ao número de nós visitados, percebe-se uma redução expressiva conforme a heurística utilizada se torna mais informativa: o A* com a distância de Hamming visitou em média 12402,22 nós, enquanto o A* com a distância de Manhattan reduziu esse número para 1088,67, e o A* com Conflito Linear obteve o menor valor entre as variantes do A*, com apenas 572,89 nós visitados em média. Esse resultado confirma a relação de dominância entre as heurísticas discutida na Seção 5, em que $h_{\text{Hamming}}(n) \leq h_{\text{Manhattan}}(n) \leq h_{\text{Conflito Linear}}(n)$.

A Busca Gulosa, por sua vez, apresentou o menor número de nós visitados entre todos os algoritmos testados (variando de 311,22 com Hamming a 55,22 com Conflito Linear), o que é coerente com sua estratégia de expandir sempre o estado que parece mais próximo do objetivo, sem considerar o custo do caminho já percorrido. Entretanto, essa eficiência tem um custo: a Busca Gulosa não garante a solução ótima, apresentando profundidades médias de solução maiores que o ótimo (62,78 com Hamming, 45,44 com Manhattan e 28,33 com Conflito Linear), confirmando seu caráter não ótimo.

Em relação ao tempo de execução, o Custo Uniforme apresentou o maior tempo médio (466,85 ms) entre os algoritmos não informados, apesar de visitar um número de nós semelhante ao BFS, possivelmente devido ao custo computacional adicional do gerenciamento da fila de prioridade. Entre as variantes do A*, observa-se uma redução expressiva no tempo médio à medida que a heurística se torna mais informativa, passando de 90,62 ms (Hamming) para 7,61 ms (Manhattan) e 5,32 ms (Conflito Linear). Já a Busca Gulosa apresentou os menores tempos médios de execução entre todos os algoritmos testados, à custa da perda de otimalidade da solução.

6 ANÁLISE COMPARATIVA

6.1 Algoritmo mais rápido, menor número de nós visitados e consistência dos resultados

Entre os algoritmos não informados, o BFS foi o mais rápido em média (228,93 ms), seguido pelo DFS (419,33 ms) e pelo Custo Uniforme, que apresentou o maior tempo médio (466,85 ms), apesar de visitar um número de nós semelhante ao BFS (73171,22 contra 75226,56). Esse maior tempo do Custo Uniforme provavelmente se deve ao custo computacional adicional do gerenciamento da fila de prioridade, em comparação à fila simples (FIFO) utilizada pelo BFS.

Considerando todos os algoritmos implementados, a Busca Gulosa com a heurística de Conflito Linear foi a configuração mais rápida (1,00 ms em média) e também a que visitou o

menor número de nós (55,22 em média). Esse resultado é coerente com a natureza da Busca Gulosa, que prioriza estados aparentemente mais próximos do objetivo sem considerar o custo do caminho já percorrido, explorando assim uma fração muito menor do espaço de estados.

Quanto à consistência dos resultados, observa-se uma variação relevante entre o menor e o maior tempo registrado em quase todos os algoritmos, por exemplo, o Custo Uniforme variou de 0,08 ms a 1123,01 ms, e o A* com Hamming variou de 1,80 ms a 344,25 ms. Essa diferença indica que o desempenho de cada algoritmo depende fortemente da instância de teste (da configuração inicial do tabuleiro), e não apenas do algoritmo em si: instâncias mais distantes do objetivo tendem a exigir a exploração de um número muito maior de estados, especialmente nos algoritmos não informados e nas heurísticas menos informativas.

6.2 Impacto das heurísticas no desempenho do A*

As três heurísticas tiveram um impacto significativo no desempenho do A*, tanto em número de nós visitados quanto em tempo de execução, mantendo a mesma profundidade média de solução (19,00 movimentos) nas três variantes, ou seja, todas preservaram a otimalidade da solução, como esperado para heurísticas admissíveis.

A heurística de Hamming foi a menos eficiente das três, com 12402,22 nós visitados em média e 90,62 ms de tempo médio, por considerar apenas se cada peça está ou não na posição correta, sem levar em conta a distância real até essa posição. A distância de Manhattan reduziu drasticamente esses valores (1088,67 nós e 7,61 ms em média), por fornecer uma estimativa mais próxima do custo real, já que considera o número de movimentos necessários para cada peça alcançar sua posição correta. Por fim, o Conflito Linear foi a heurística mais eficiente entre as três, com apenas 572,89 nós visitados e 5,32 ms em média, pois refina a distância de Manhattan ao identificar pares de peças que precisam se desviar de sua linha ou coluna, fornecendo uma estimativa ainda mais próxima do custo real e, por isso, reduzindo ainda mais o número de estados explorados pelo A* até encontrar a solução ótima.

6.3 Diferenças nos caminhos encontrados entre os algoritmos

Os algoritmos BFS, Custo Uniforme e A* (nas três variantes de heurística) encontraram caminhos de mesmo tamanho médio (19,00 movimentos), ou seja, todos esses algoritmos garantiram a solução ótima para as instâncias testadas. Isso é esperado, já que o custo de cada movimento no 8-puzzle é uniforme, e tanto o BFS quanto o Custo Uniforme garantem otimalidade nesse cenário; o A*, por sua vez, garante otimalidade por utilizar heurísticas admissíveis.

Já o DFS e a Busca Gulosa encontraram caminhos significativamente maiores que o ótimo. O DFS apresentou um número médio de movimentos extremamente alto (86379,89), pois explora o espaço de estados em profundidade, podendo se afastar muito do objetivo antes

de eventualmente alcançá-lo, sem qualquer garantia de otimalidade. A Busca Gulosa, embora muito mais eficiente que o DFS, também não garantiu soluções ótimas, com profundidades médias variando entre 28,33 (Conflito Linear) e 62,78 (Hamming) movimentos, isso ocorre porque a Busca Gulosa não considera o custo do caminho já percorrido ($g(n)$), guiando-se apenas pela heurística ($h(n)$), o que pode levá-la a seguir por caminhos mais longos que parecem promissores localmente.

6.4 Recomendação de uso para cada algoritmo

Com base nos resultados obtidos, recomenda-se o uso do A* com a heurística de Conflito Linear (ou, alternativamente, com a distância de Manhattan) em situações em que se deseja garantir a solução ótima com o menor custo computacional possível, sendo a escolha mais equilibrada entre qualidade da solução e eficiência.

O BFS pode ser recomendado em cenários nos quais a implementação de uma heurística não é viável ou desejada, mas ainda se deseja garantir a solução ótima, sendo uma alternativa mais simples que o A*, embora menos eficiente.

O Custo Uniforme, por apresentar desempenho semelhante (ou pior) ao BFS no contexto específico do 8-puzzle, problema com custo uniforme por passo, não traz vantagens práticas aqui, sendo mais indicado para problemas em que os custos das ações não são uniformes.

A Busca Gulosa é recomendada em situações em que a velocidade de resposta é mais importante do que a qualidade da solução, ou quando se aceita uma solução apenas razoavelmente boa (não necessariamente ótima), sendo especialmente útil em conjunto com heurísticas mais informativas, como o Conflito Linear.

Por fim, o DFS não é recomendado para a resolução do 8-puzzle, dado seu desempenho extremamente inferior tanto em tempo de execução quanto em qualidade da solução encontrada, sendo mais adequado a problemas em que a profundidade da árvore de busca é conhecida e limitada, ou em que o objetivo é apenas verificar a existência de uma solução, sem necessidade de otimalidade.

7 DIÁRIO DE DECISÕES

7.1 Maior dificuldade técnica encontrada e como foi resolvida

Não encontramos muitas dificuldades durante a implementação. Como BFS, DFS, Custo Uniforme, Busca Gulosa e A* compartilham uma estrutura geral semelhante, baseada na expansão de estados a partir de uma fronteira, era possível cometer pequenos desvios ao reutilizar trechos de código, comprometendo o comportamento esperado de cada algoritmo. No entanto, ao analisar cada caso individualmente, os conflitos identificados eram resolvidos rapidamente.

A principal dificuldade ocorreu na exibição do caminho completo gerado pela DFS, pois esse caminho podia ultrapassar 150 mil movimentos. Inicialmente, utilizamos uma função recursiva para exibir o caminho, porém enfrentamos problemas de stack overflow nesse algoritmo. Posteriormente, implementamos uma versão iterativa, que resolveu o problema.

Essa dificuldade foi resolvida por meio de testes de depuração, nos quais incluímos saídas ao longo do código para identificar o trecho que apresentava problemas. Ao localizar a falha na exibição do caminho e adicionar um contador para analisar o congelamento do programa, identificamos que, após aproximadamente 32 mil chamadas recursivas, o programa deixava de responder. Isso nos levou a adaptar a solução para uma função iterativa, que inverte o vetor contendo os caminhos e os exibe de forma sequencial. Diferentemente da implementação anterior, essa abordagem evita acessar os caminhos recursivamente até encontrar o estado inicial antes de exibir a sequência.

7.2 O que foi tentado e não funcionou durante o desenvolvimento

Não houve abordagens ou implementações que tenham sido tentadas e posteriormente descartadas durante o desenvolvimento, com exceção do problema descrito anteriormente. O processo seguiu de forma direta até as soluções implementadas, com o esforço concentrado principalmente em garantir a confiabilidade da coleta de dados.

7.3 Melhorias e explorações futuras, caso houvesse mais tempo

Com mais tempo disponível, seria útil complementar o código para que, ao final da execução, fosse criado um arquivo em formato XLS contendo as métricas referentes ao algoritmo em uma tabela. Essa melhoria, aliada a uma opção que possibilitasse executar todos os algoritmos e suas heurísticas a partir de uma única entrada, melhoraria a experiência do usuário, facilitando a comparação dos resultados por meio da tabela e eliminando a necessidade de iniciar a solução manualmente para cada heurística.

interessante aprimorar a visualização do problema do 8-puzzle por meio do desenvolvimento de uma interface gráfica que permitisse ao usuário selecionar o algoritmo de busca desejado e acompanhar sua execução de forma interativa. Além disso, poderiam ser investigadas e implementadas outras heurísticas para os algoritmos de busca informada, possibilitando a realização de uma comparação mais abrangente entre diferentes estratégias heurísticas quanto ao desempenho e à qualidade das soluções obtidas.

Referências

cp-algorithms.com. *Breadth First Search*. Disponível em: <https://cp-algorithms.com/graph/breadth-first-search.html>. Acesso em: 26 jun. 2026.

cp-algorithms.com. *Depth First Search*. Disponível em: <https://cp-algorithms.com/graph/depth-first-search.html>. Acesso em: 26 jun. 2026.

cp-algorithms.com. *Dijkstra - finding shortest paths from given vertex*. Disponível em: <https://cp-algorithms.com/graph/dijkstra.html>. Acesso em: 26 jun. 2026.

OLIVEIRA, Gabriel Pereira de. *Slides da disciplina de Inteligência Artificial*. PUC Minas, [s.d.]. Material didático disponível na plataforma Canvas da PUC Minas.